김동영 FRONTEND DEVELOPER

React, TypeScript

010-5357-9878 | Dongykim414@gmail.com

오늘의웹툰의 첫 프론트엔드 개발자로 합류해 웹툰 연재 플랫폼의 초기 MVP 부터 출시 후 운영까지 참여하고, 작품과 마케팅 데이터를 활용한 B2B 분석 서비스와 사내 운영 도구를 개발했습니다.

빠르게 필요한 기능을 출시한 뒤 사용자 문의와 행동 로그를 확인해 사용 중 막히는 지점을 보완했습니다. 반복되는 데이터 요청이나 복잡한 화면 요구는 공통 구조로 정리했고, 새로운 구현 방식을 선택할 때는 개발 비용뿐 아니라 이후의 변경과 운영 영향도 함께 고려했습니다.

경력 및 주요 프로젝트

오늘의웹툰 프론트엔드 개발자 | 2021.05 – 2025.02

웹툰 연재 플랫폼 B2C | 2022.08 – 2023.09

작가가 작품과 회차를 등록하고 공모전 및 분석을 신청하며, 독자는 작품을 탐색하고 열람하는 서비스를 구축했습니다. 가입 중간 실패와 재신청 문제를 백엔드와 함께 개선했고, Amplitude 와 CloudWatch 로그로 모바일 메인 이탈을 확인해 연속 탐색형 피드로 개편했습니다. 여러 화면의 작품 조회는 Recoil 기반 공통 훅으로 통합했습니다.

웹툰 메트릭 (Metric) B2B SaaS | 2023.10 – 2024.09

작품과 광고의 매출, 순위, 독자 지표를 비교하는 대시보드와 유료 분석 리포트, 광고 운영 화면을 개발했습니다. 서로 다른 지표에 대응하는 D3 와 React 기반 공통 차트를 분석 대시보드 및 리포트에 적용하고, 운영 제품을 그대로 활용하는 샘플 데이터 기반 고객용 데모의 구현 방식을 제안하고 개발했습니다.

운영 백오피스 및 공통 UI 사내 운영 및 서비스 공통

한글 초성 및 불완전 음절 검색, Redux 기반 작품 조회 캐시와 공모전 운영 도구를 구현했습니다. Toast, Loading, 중첩 Drawer 를 공통화해 3 개 자사 서비스에서 재사용하고, PR 리뷰와 인턴 온보딩에도 참여했습니다.

기술 및 추가 경험

React, TypeScript, JavaScript, Recoil, Redux, D3.js, visx, Axios, Material UI, SCSS

React Portal, Context API, Chart.js, AWS S3 / CloudWatch, Git, Amplitude, Puppeteer

한국방송통신대학교 컴퓨터과학 학사 | 정보처리기사

기타: Unity / C# 기반 2.5D 웹툰 IP 모바일 게임의 iOS 와 Android 출시 및 업데이트

프로젝트에서 실제로 한 일

앞 장에서 소개한 경험 중 문제를 확인하고 구현 방식을 결정했던 사례를 구체적으로 정리했습니다.

가입 과정이 중간에 멈춰도 다시 신청할 수 있도록

웹툰 연재 플랫폼 | React, Axios | 입력 검증, 오류 추적, 요청 흐름

회원가입 한 번의 제출 과정에서 계정, 작품, 회차, 분석 신청이 순차적으로 생성됐습니다. 뒤 단계에서 통신이 실패하면 이미 생성된 데이터가 남아 같은 정보로 다시 가입하기 어려웠고, 필수 이미지를 누락하면 버튼만 비활성화돼 원인을 알 수 없다는 문의도 있었습니다. 입력 누락 항목을 명시적으로 안내하고, 생성 단계별 요청을 분리해 오류 로그를 남겼습니다. 백엔드와 실패 시 중간 생성 데이터의 정리 및 반복 요청 처리 방식을 협의했으며, 프론트에서는 요청 완료 전 재제출을 막았습니다. 이후 관련 가입 실패 사례와 문의가 감소했습니다.

필요한 작품 데이터만 요청하고 화면 간에 재사용하도록

웹툰 연재 플랫폼 | React, Recoil, Custom Hook | 데이터 요청, 캐시, 갱신

홈, 작품 상세, 구독 UI 의 여러 컴포넌트가 같은 작품 데이터를 각각 요청했습니다. 화면별 요청 로직을 고치는 대신 공통 데이터 계층이 필요하다고 판단했고, 기존 Redux 방식의 구현 부담과 React Query 도입 비용을 검토해 Recoil 과 커스텀 훅으로 조회를 묶었습니다. 서비스와 작품을 함께 구분하는 복합 식별키를 기준으로 이미 요청한 작품을 걸러 미조회 대상만 가져오고, 작품 수정이나 구독 변경 뒤에는 명시적인 갱신 함수를 호출했습니다. 이로써 같은 화면의 중복 작품 요청을 통합하고 홈, 상세, 구독 화면에서 응답을 재사용했습니다.

서로 다른 지표를 하나의 차트 인터페이스로 표현하도록

웹툰 메트릭 | React, TypeScript, D3.js | 차트 설계, 렌더링 최적화

순위는 1 위가 위에 표시되어야 했고, 값이 좁은 범위에 몰린 지표는 전체 축을 표시하면 차이를 읽기 어려웠습니다. 지표별로 별도 차트를 늘리기보다 축 방향, 표시 구간, 영역 표현을 설정으로 제어하도록 설계했습니다. D3 에는 scale, tick, SVG path 계산을, React 에는 SVG 렌더링과 상호작용을 맡겨 DOM 제어의 충돌을 피했습니다. 이후 모든 차트에 전체 데이터가 전달되던 구조를 필요한 데이터만 전달하도록 정리하고, 일부 계산에 useMemo 를 적용해 결과를 재사용했습니다. 공통 차트 구조를 분석 대시보드와 리포트에 적용했습니다.

운영 화면을 복제하지 않는 고객용 데모를 만들도록

웹툰 메트릭 | Axios Interceptor, Custom Adapter | 대안 비교, 운영 영향

서비스를 구매하기 전 실제 사용 화면을 체험하고 싶다는 문의가 반복됐습니다. 별도 데모 사이트는 화면 변경 시 동기화 부담이 있고, 테스트 계정은 운영 데이터에 영향을 줄 수 있어 다른 방식을 검토했습니다. 기존 UI 와 API 호출 계약을 유지하고, Axios interceptor 와 custom adapter 에서 데모 대상 요청만 샘플 응답으로 전환하는 방식을 제안하고 구현했습니다. 광고 신청처럼 실제 데이터를 변경할 수 있는 요청은 운영 서버에 전달되지 않도록 제한했습니다. 고객이 제품 화면을 체험하면서도 별도의 데모 UI 를 유지보수하지 않아도 되는 구조를 만들었습니다.

그 밖의 구현 React Portal 과 Context 를 활용한 중첩 Drawer 순서 및 종료 동작 관리, 운영 백오피스 초성 검색, 모바일 피드 탐색 UI